

# Object-Oriented Programming

## Assignment #1 Da Vinci Code

담당교수: 윤은영

학번: 20180400

이름: 신민석

명예서약(Honor code)

"나는 이 프로그래밍 과제를 다른 사람의 부적절한 도움 없이 완수하였습니다."

### 목차

|                                     |    |
|-------------------------------------|----|
| Object-Oriented Programming .....   | 1  |
| 1. 프로그램 개요 .....                    | 2  |
| # 목표 .....                          | 2  |
| # 문제 이해 .....                       | 2  |
| 2. 프로그램 구조 및 구현 설명 .....            | 2  |
| # 요약 .....                          | 2  |
| # 상세 .....                          | 3  |
| # Flow chart for int goGame() ..... | 5  |
| 3. 프로그램 실행 및 예제 시연 .....            | 5  |
| # 컴파일 및 실행 .....                    | 6  |
| # 리눅스 환경 .....                      | 9  |
| 4. Discussion & Improvements.....   | 10 |
| 5. References .....                 | 10 |

## 1. 프로그램 개요

## # 목표

이번 프로그램에서 자료형에 대해 공부하며, 구조체를 통한 사용자 정의 자료형의 사용을 숙달한다. 또한 Linked list의 데이터 manipulation을 구현한다. 또한 참조자와 같은 C++의 새로운 몇몇 기능들 또한 이용하도록 하였다. 본 assignment에서 class는 사용하지 않았다. 본 보고서 전체에 걸쳐 유저=usr=플레이어이고, pc=컴퓨터이다.

## # 문제 이해

이번 문제에서는 다빈치 코드 보드게임을 컴퓨터로 구현하는 것을 목표로 한다. 대부분의 규칙은 원래의 룰과 비슷하나, 일부 차이점이 있다.

- 조커 타일을 사용하지 않는다
- 턴을 임의로 넘길 수 없다
- 패가 다 떨어졌을 경우 비공개 타일 중 가장 왼쪽에 있는 타일을 공개한다.

컴퓨터는 눈에 보이는 숫자를 대답하지 않는 수준의 지능을 지니어 플레이어가 승리하기 쉽다.

## 2. 프로그램 구조 및 구현 설명

## # 요약

본 프로그램에서 게임을 진행하는데 필요한 다양한 기능들을 분할하여 함수로 작성하였으며 다음 목록과 같은 관계를 가진다.

|           |           |                      |
|-----------|-----------|----------------------|
| main      |           | 프로그램의 구조가 시작된다       |
| Category1 | prtMenu   | 프로그램의 메뉴를 보여준다       |
|           | prtManual | 게임의 설명을 출력한다         |
|           | goGame    | 게임이 진행된다             |
| Category2 | countTile | 패의 개수를 조건하에 센다       |
|           | plushTile | 게임이 끝나고 패의 메모리를 해제한다 |
|           | genTile   | 게임 시작 시 패를 생성한다      |
|           | initTile  | 패를 처음 세팅한다           |
|           | getTile   | 패를 가져온다              |
|           | pcTurn    | Pc의 턴이 된다            |
|           | pcGuess   | Pc가 추리한다             |
|           | usrTurn   | 유저가 턴이 된다            |
|           | usrGuess  | 유저가 추리한다             |

## # 상세

## ● 공통사항:

- Rtnvalue: 이는 프로그램에서 예외적이고 원치 않는 상황이 발생할 경우 0 이외의 값을 리턴하여 프로그램 종료를 일으킨다. 본 프로그램에서 3자리의 int인 경우는 일어나면 안 되는 예외상황이다. -1, 10000은 정상 상황이다.
- `tile*` position: 반복문에서 `i`가 사용되는 용례와 비슷하며 linked list 탐색시에 현재 위치를 잠시 기억한다.
- 타일 구조체는 linked list 로 구현되었으며, "`game.h`"에 정의되어 있다. 직관적인 변수 이름이다.

```
struct tile {
    bool visibility;
    bool white;
    int number;
    tile* next;
    tile* prev;
}; //linked list struct
```

- `tile*& trunk`, `tile*& pc`, `tile*& usr`: `trunk`는 바닥에 깔려 있는 타일이다. `pc`, `usr`는 각각 이 가진 타일 덩어리이다. `pc`는 컴퓨터 `usr`는 플레이어를 의미한다.

## ● Category 2 함수 설명:

- `int countTile(const tile* head, int mode, bool data)`  
타일 linked list의 개수를 세는 함수이다. `int`를 반환하며, 개수만 세므로 `const tile*`자료형을 받아 사용한다. `int mode`는 타일을 세는 조건을 의미하며 `0=all, 1=white, 2=visibility`의 할당을 가진다. `bool data`는 `mode`가 1,2 일 때 해당 데이터 조건을 의미한다.
- `tile* genTile()`  
게임 시작시에 동적 할당을 통하여 타일 linked list를 생성하는 함수이다. 생성된 타일 linked list는 sorting 되어있는 형태를 가지며, 생성된 linked list의 head인 `tile* trunk`를 반환한다.
- `void plushTile(tile* head)`  
게임 종료시에 동적 할당된 타일 linked list를 모두 할당 해제한다. `for` 반복문을 사용해 타일 하나씩 제거한다.
- `int getTile(tile*& trunk, tile*& tgt, int control, tile*& recent)`  
`trunk`에서 `tgt`(타겟, `pc` or `usr`)으로 타일을 가져온다. 이 때 가져오는 조건들이 있는데, `control` 값에 따라 결정된다. `control 0=black, 1=white, 2=usr choice, 3=random color`의 조건이 있으며, `recent`는 마지막으로 가져온 타일을 가리키도록 된다. 만약 `trunk`에 타일이 없

다면 10000을 리턴하며, 또는 특정 색상 타일만 있다면 알아서 그 타일을 가져오게 된다.

- `int initTile(tile*& trunk, tile*& pc, tile*& usr)`  
tile deck을 게임 시작시에 initialize 하는 함수이다. Pc와 usr 각각 4개씩 타일이 생기게 되며, usr는 가져올 타일의 색 조합을 선택할 수 있다. 이 함수는 `int getTile` 함수를 여러 번 호출하는 구성이다.
- `int pcTurn(tile*& trunk, tile*& pc, tile*& usr, int& win)`  
pc의 턴을 진행하는 함수이다. Pc는 `getTile` 함수를 통해 trunk 타일의 상태를 보고 랜덤으로 타일을 가져와 자신의 텍에 끼워 넣는다. 타일이 없다면(if `getTile` returns 10000) 안 가져온다. 그 후 `pcGuess` 함수를 통하여 오답시까지 추리를 반복한다. 매 추리마다 승자/패자 여부를 체크한다. 이는 do while 문에 작성되어 있다.
- `int pcGuess(tile*& trunk, tile*& pc, tile*& usr, tile*& recent)`  
pc가 usr의 타일을 추리하는 함수이다. 여기서 usr의 타일을 visibility 관계없이 랜덤으로 뽑는다. 만약 visible 한 타일을 뽑았을 경우 visibility=0이 될 때까지 반복한다. 매번 추리 후에 `pcTurn`과 `usrTurn`에서 viability count check를 하여 invisible 타일이 한 개 이상 존재하므로 무한 반복은 없어야 정상이다. 추측 대상은 `tile* obj` 로 기록한다.  
배열 `int guesslist[24]`는 추측해도 되는 숫자들의 목록이다. 이는 안보이는 특정 색 타일의 숫자를 모아서 생성한다. 전체 숫자에서 보이는 숫자를 빼는 것이 아닌 안보이는 숫자를 합하는 식으로 작동한다. 그 후 상대 타일과의 비교를 통하여 정답 오답 여부를 리턴값으로 전달해준다. (-1정답, 0오답)
- `int usrTurn(tile*& trunk, tile*& pc, tile*& usr, int& win)`  
usr의 턴을 진행하는 함수이다. usr는 `getTile` 함수를 통해 trunk 타일의 상태를 보고 원하는 색상의 타일을 가져와 자신의 텍에 끼워 넣는다. 타일이 없다면(if `getTile` returns 10000) 안 가져온다. 그 후 `usrGuess` 함수를 통하여 오답시까지 추리를 반복한다. 매 추리마다 승자/패자 여부를 체크한다. 이는 do while 문에 작성되어 있다.
- `int usrGuess(tile*& trunk, tile*& pc, tile*& usr, tile*& recent)`  
usr가 pc의 타일을 추리하는 함수이다. Pc의 타일 위치와 추측숫자를 입력 받는다. 추측 대상은 `tile* obj` 로 기록한다. 그 후 상대 타일과의 비교를 통하여 정답 오답 여부를 리턴값으로 전달해준다. (-1정답, 0오답)

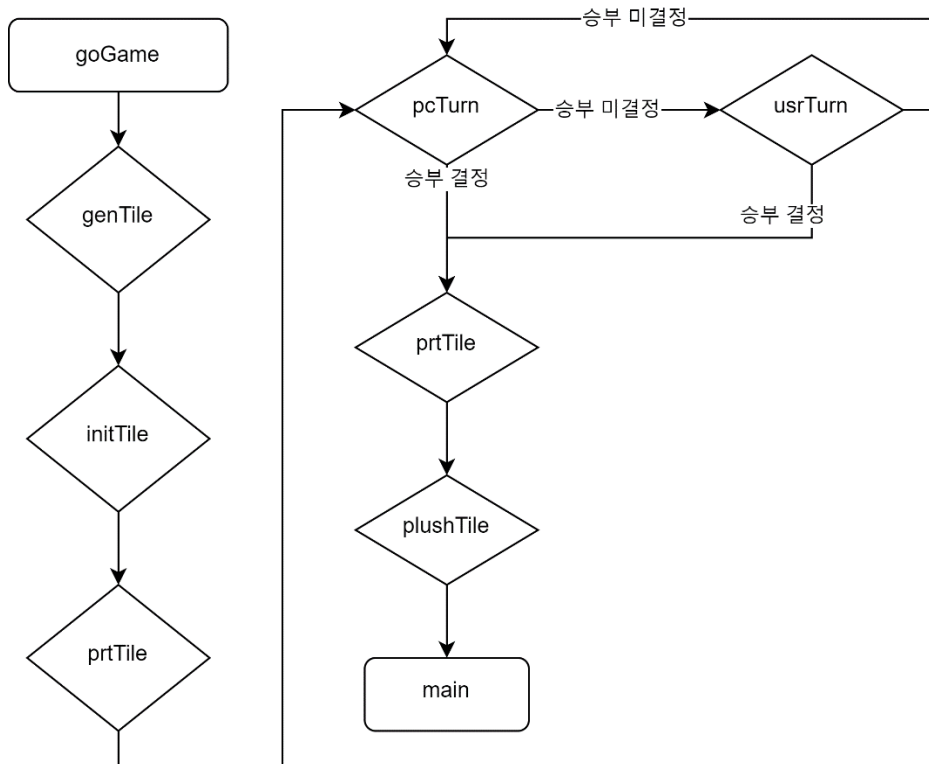
● Category 1 함수 설명:

- `int goGame()`  
게임을 진행하는 함수이다. Category 2의 함수들을 호출하며 게임을 시작하고 끝내는 부분까지 담당한다. `gentile` 함수를 통해 타일을 생성하고, `initTile` 함수를 통해 초기 판을 세팅하며, 그 후 타일 출력과 pc와 usr 사이의 턴 반복을 주관한다. 만약 턴이 끝나서 승부가 난다면, 승리 메시지를 출력한 후 `plushTile`을 통해 동적 할당을 해제한 후 게임을 끝낸다. `tile*` 들

의 본체가 생성되는 곳으로 숫자타일은 이 함수 하위에서 존재한다. 만약 일어나서는 안되는 사유로 비정상적으로 종료될 때 동적 메모리 할당을 해제한다.

- `void prtManual()`  
cout을 통해 게임의 룰을 설명하는 문자열을 출력한다.
- `void prtMenu()`  
cout을 통해 메뉴판을 화면에 출력한다.
- `int main()` 함수 설명  
Category 1의 함수들을 메인 메뉴의 사용자 입력에 따라 호출하는 역할과, 하위 함수들의 리턴값을 받아 최종적으로 리턴해주는 역할을 한다.

# Flow chart for `int goGame()`



### 3. 프로그램 실행 및 예제 시연

본 프로그래밍 과제는 윈도우 vs 2022 환경에서 진행하였다. 코드 파일은 함수별로 분할하지는 않고 2~4개의 비슷한 성격의 함수끼리 모아져 있으며, 코드 파일 리스트는 다음과 같다.

|          |         |           |        |          |        |          |
|----------|---------|-----------|--------|----------|--------|----------|
| main.cpp | menus.h | menus.cpp | game.h | game.cpp | turn.h | turn.cpp |
|----------|---------|-----------|--------|----------|--------|----------|

## # 컴파일 및 실행

```
C:\Users\mshks\source\repos\ConsoleApplication2\Debug\ConsoleApplication2.exe
[ 다빈치 코드 ]
=====
1.      게임 설명
2.      게임 시작
3.      종료
=====
선택:
1
===== 게임 설명 =====
다빈치코드는 숫자가 쓰여 있는 타일을 나눠갖고, 다른 사람의 타일에 쓰인 숫자를 맞히는 게임이다.
각 타일은 숫자 0~11로 구성되며, 흰색 또는 검정색을 띤다. (조커는 없다고 가정한다.)
이번 프로그래밍 과제에서는 사용자와 컴퓨터간의 게임, 즉 2인 플레이를 가정한다.
각 플레이어는 게임을 시작할 때 원하는 4개의 타일을 숫자를 보지 않고 가져간다.
자신의 패에 있는 타일은 가작 작은 수 부터 가장 큰 수까지 순서대로 배열해야 한다.
검은색과 흰색이 같은 숫자일 경우 검은색을 먼저 오도록 놓는다.
컴퓨터가 첫 턴을 가져가는 것으로 한다.
자신의 차례에는 뒤집어진 타일을 하나 가져와 규칙대로 자기 패에 끼워넣는다.
이때 가져온 타일이 조커타일이라면 자신이 원하는 위치에 집어 넣을 수 있다.
상대 플레이어의 타일 하나를 지목해 숫자를 맞혀야 한다.
말한 숫자가 맞다면 상대는 그 타일을 보이도록 놓혀 공개해야 한다.
맞혔을 경우, 다른 타일의 숫자를 계속해서 맞혀본다. (또는 차례를 넘길 수 있다.)
틀렸을 경우, 그 차례에 가져갔던 타일을 놓혀 숫자를 공개해야 한다.
숫자가 모두 드러난 사람은 패한다.
=====
[ 다빈치 코드 ]
=====
1.      게임 설명
2.      게임 시작
3.      종료
=====
선택:
-
```

프로그램을 실행하면 3개의 메뉴가 뜨고, 1번을 선택했을 경우 게임 설명이 나온다.

```
[ 다빈치 코드 ]
=====
1.      게임 설명
2.      게임 시작
3.      종료
=====
선택:
2
가져올 타일의 색 별 개수를 정하세요. (흑 백 순서, 흑+백=4):
2 3
가져올 타일의 색 별 개수를 정하세요. (흑 백 순서, 흑+백=4):
2 2
=====
남은 타일 || □:8 ■:8
=====
==컴퓨터 텍==
[1: □??] [2: □??] [3: ■??] [4: ■??]
==플레이어 텍==
[1: ■2] [2: □3] [3: ■9] [4: □11]
```

게임 시작을 누를 경우 게임이 시작된다. 가져올 타일의 개수가 맞지 않으면 다시 입력받으며, 초기 텍 세팅이 완료된 모습이다.

```

컴퓨터의 차례입니다

컴퓨터가 랜덤 타일을 고르고 있습니다...
컴퓨터가 백색타일을 골랐습니다.

=====
남은 타일 || □:8 ■:7
=====

==컴퓨터 덱==
[1: □??] [2: ■??] [3: □??] [4: ■??] [5: ■??]

==플레이어 덱==
[1: ■2] [2: □3] [3: ■9] [4: □11]

컴퓨터가 추리를 시작합니다.
컴퓨터가 3번째 위치의 숫자가 11라고 추리합니다.
틀렸습니다!
컴퓨터의 백색 타일 4번이 공개됩니다.

=====
남은 타일 || □:8 ■:7
=====

==컴퓨터 덱==
[1: □??] <2: ■4> [3: □??] [4: ■??] [5: ■??]

==플레이어 덱==
[1: ■2] [2: □3] [3: ■9] [4: □11]

```

컴퓨터가 플레이어의 타일을 잘못 추리하여 방금 뽑은 타일이 공개되었다.

```

플레이어의 차례입니다

가져오고 싶은 타일의 색을 고르세요. (흑:0 백:1):
1
가져온 타일은 백색타일1번입니다.

=====
남은 타일 || □:8 ■:6
=====

==컴퓨터 덱==
[1: □??] <2: ■4> [3: □??] [4: ■??] [5: ■??]

==플레이어 덱==
[1: ■1] [2: ■2] [3: □3] [4: ■9] [5: □11]

플레이어가 추리를 시작합니다.
추리하고 싶은 위치와 숫자를 입력해주세요.
1 2
플레이어가 1번째 위치의 숫자가 2라고 추리합니다.
정답입니다!
컴퓨터의 흑색 타일 2번이 공개됩니다.

=====
남은 타일 || □:8 ■:6
=====

==컴퓨터 덱==
<1: □2> <2: ■4> [3: □??] [4: ■??] [5: ■??]

==플레이어 덱==
[1: ■1] [2: ■2] [3: □3] [4: ■9] [5: □11]

플레이어가 추리를 시작합니다.
추리하고 싶은 위치와 숫자를 입력해주세요.
3 5
플레이어가 3번째 위치의 숫자가 5라고 추리합니다.
틀렸습니다!
플레이어의 백색 타일 1번이 공개됩니다.

```

플레이어가 타일을 한번 맞추었고 두 번째 추리에서는 오답이여서 가져온 타일이 공개된다.

이후 중간 플레이 과정은 너무 길어지므로 생략한다.

```

=====
남은 타일 || □:6 ■:4
=====

==컴퓨터 덱==
<1: □2> <2: ■3> <3: □4> <4: ■4> <5: □6> [6: ■??] <7: ■10>

==플레이어 덱==
[1: ■0] <2: ■1> [3: ■2] [4: □3] <5: □5> [6: ■9] [7: □11]

플레이어가 추리를 시작합니다.
추리하고 싶은 위치와 숫자를 입력해주세요.
6 7
플레이어가 6번째 위치의 숫자가 7라고 추리합니다.
정답입니다!
컴퓨터의 백색 타일 7번이 공개됩니다.

=====
남은 타일 || □:6 ■:4
=====

==컴퓨터 덱==
<1: □2> <2: ■3> <3: □4> <4: ■4> <5: □6> <6: ■7> <7: ■10>

==플레이어 덱==
[1: ■0] <2: ■1> [3: ■2] [4: □3] <5: □5> [6: ■9] [7: □11]

플레이어의 승리입니다
=====
남은 타일 || □:6 ■:4
=====

==컴퓨터 덱==
<1: □2> <2: ■3> <3: □4> <4: ■4> <5: □6> <6: ■7> <7: ■10>

==플레이어 덱==
[1: ■0] <2: ■1> [3: ■2] [4: □3] <5: □5> [6: ■9] [7: □11]

게임이 종료되었습니다. 메뉴로 돌아갑니다

[ 다빈치 코드 ]
=====
1.   게임 설명
2.   게임 시작
3.   종료
=====
선택:

```

컴퓨터의 타일이 모두 공개되어 플레이어의 승리가 되어 메인 메뉴로 돌아간 모습이다.

이 외에 바닥에 뽑을 타일이 없는 경우에도 정상적으로 가장 왼쪽 비공개 타일을 공개하는 것으로 작동함을 확인하였다.



## # 리눅스 환경

```

power@DESKTOP-C641PQL:~/proj1$ make
g++ -c game.cpp -o game.o
g++ -c turn.cpp -o turn.o
g++ -c menus.cpp -o menus.o
g++ -c main.cpp -o main.o
g++ game.o turn.o menus.o main.o -o proj1.out
power@DESKTOP-C641PQL:~/proj1$ ./proj1.out
[ 다빈치 코드 ]
=====
1.      게임 설명
2.      게임 시작
3.      종료
=====
선택:
1
===== 게임 설명 =====
다빈치코드는 숫자가 쓰여 있는 타일을 나눠갖고, 다
각 타일은 숫자 0~11로 구성되며, 흰색 또는 검정색을
이번 프로그래밍 과제에서는 사용자와 컴퓨터간의 게
각 플레이어는 게임을 시작할 때 원하는 4개의 타일을
자신의 패에 있는 타일은 가작 작은 수 부터 가장 큰
검은색과 흰색이 같은 숫자일 경우 검은색을 먼저 오
컴퓨터가 첫 턴을 가져가는 것으로 한다.
자신의 차례에는 뒤집어진 타일을 하나 가져와 규칙
이때 가져온 타일이 조커타일이라면 자신이 원하는
상대 플레이어의 타일 하나를 지목해 숫자를 맞춰야
말한 숫자가 맞다면 상대는 그 타일을 보이도록 놓혀
맞혀올 경우 다른 타일의 숫자를 계속해서 맞춰본다

```

리눅스 환경에서도 잘 실행된다.

```

CPP      = g++
OBJ      = game.o turn.o menus.o main.o
LINKOBJ  = game.o turn.o menus.o main.o
BIN      = proj1.out

.PHONY: clean

$(BIN): $(OBJ)
    $(CPP) $(LINKOBJ) -o $(BIN)

game.o: game.cpp
    $(CPP) -c game.cpp -o game.o

turn.o: turn.cpp
    $(CPP) -c turn.cpp -o turn.o

menus.o: menus.cpp
    $(CPP) -c menus.cpp -o menus.o

main.o: main.cpp
    $(CPP) -c main.cpp -o main.o

clean:
    rm $(OBJ) $(BIN)

~
~
~
"makefile" [noeol][dos] 25L, 423C

```

makefile은 위와 같다.

#### 4. Discussion & Improvements

- 비주얼 스튜디오 소스파일의 인코딩이 파일마다 제각각이라 리눅스 상에서 컴파일 할 경우 한글이 모두 깨져서 나왔다. 이번 과제에서 모두 utf-8 bom으로 변환하여 해결하였다.
- 메모리 누수를 점검하기 위해 visual studio 디버거를 사용하여 다양한 상황에서 테스트하였다. 결과적으로 메모리 누수는 일어나지 않음을 확인하였다.
- Rand와 time 함수를 iostream을 include 함으로써 그냥 사용할 수 있다.
- 객체 지향 언어에서 class를 사용하는 것이 필요함을 느끼게 되었다. Class를 사용하여 너무 큰 덩어리의 프로그램의 복잡성과 모호성을 개선할 수 있을 것 같다.

#### 5. References

- [CRT 라이브러리로 메모리 누수 찾기 - Visual Studio \(Windows\) | Microsoft Learn](#)
- [씹어먹는 C++ - <2. C++ 참조자\(레퍼런스\)의 도입> \(modoocode.com\)](#)